PROJECT INTERVIEW GUIDE

WearForward — Clothing Donation App

Written in simple words. Every fact comes from the real project code and documents.

How to use this guide: Read Part 1 and Part 2 first (the story and the 2-minute pitch). Then read the diagrams in Part 4 and Part 5. Then read the Questions and Answers in Part 12. In the interview, explain things in your own words. Only say what you really did or understand.

Part 1. What is this project?

In one line

WearForward is a web app where people can donate old clothes, and verified NGOs (charity organizations) can collect those clothes and give them to people in need.

The problem it solves

Many people have good clothes they do not use. NGOs need clothes for people in need. But there is no easy way to connect them. WearForward connects the donor and the NGO, and tracks the clothes from the moment they are listed until they reach the receiver.

Who uses the app? (3 roles)

Role	What they can do
Donor	Sign up, add clothing photos and details, set pickup location, see their listings, cancel a listing before pickup, get notifications, see their own dashboard.
NGO	Sign up with a registration number. If verified, browse nearby listings, claim one, mark it picked up, hand it to a receiver, or release/report failure. See their own dashboard.
Admin	Cannot sign up from the app (for safety). Created by database/seed script. Can view all listings, delete bad listings, see expired claims and NGO verification records, see the admin dashboard.

Main features

- Donor and NGO signup and login with password protection and JWT tokens.
- Donor creates a listing with many clothing items. Each item has a category, a condition and a photo.
- Donor can use “my current location” and the app finds the area name automatically.
- Verified NGOs browse open listings, filter by distance and category, then claim one.
- Each donation moves through 5 steps: Listed → Claimed → Picked up → Distributed → Delivered.
- If an NGO claims but does not pick up within 48 hours, the claim expires and the listing opens again automatically.
- In-app notifications keep donors and NGOs updated at each step.
- Dashboards with charts for donor, NGO and admin.
- Forgot password with a 6-digit code or a reset link (valid for 15 minutes).
- Privacy: NGOs cannot see the donor’s exact GPS location until they claim the listing. Only the receiver’s first name and general area are saved.

My role: Your chat history shows you built, debugged and tested this app with Claude Code in VS Code. In the interview, say clearly which parts you worked on yourself, and be ready to explain those parts in detail. If you are asked about a part you did not write, say “I understood and tested it, and here is how it works.”

Part 2. Your 2-minute pitch (say this out loud)

“I built WearForward, a clothing donation web app. A donor lists clothes with photos and a pickup location. A verified NGO can browse nearby listings and claim one. After claiming, the NGO picks up the clothes, gives them to a receiver, and the donor gets notified at every step. The listing moves through five stages: Listed, Claimed, Picked up, Distributed and Delivered.

The frontend is React with Vite. The backend is Node.js with Express, and the database is PostgreSQL. I used JWT tokens for login and bcrypt for password hashing. One interesting part is concurrency: if two NGOs try to claim the same listing at the same time, only one succeeds, because I protect it both in the code and with a database rule. I also built a background job that expires a claim after 48 hours so the clothes are not stuck. For testing, the project has 55 backend integration test suites and Playwright browser tests. For deployment, the project has a Dockerfile, a GitHub Actions pipeline and an AWS plan.”

Part 3. Tech stack and why we used each one

The “Simple reason” column gives easy reasons that fit what the code does. Use the ones that match your own thinking. It is fine to say you chose it because it is popular, well supported and good for this kind of app.

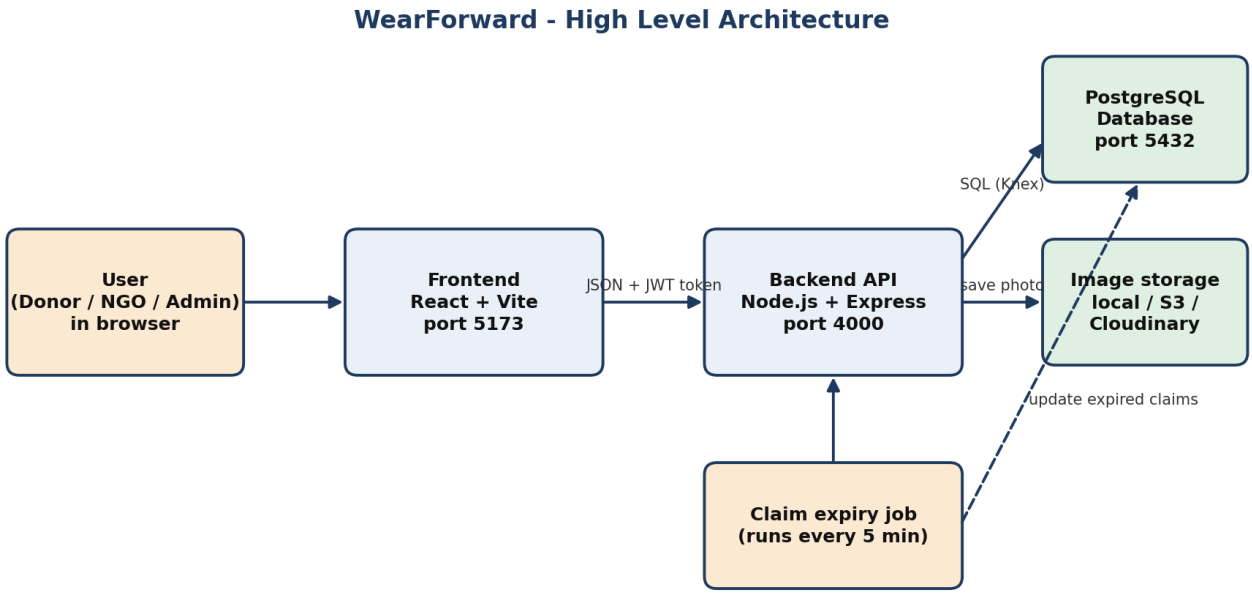
Part	Technology	What it does here	Simple reason
Frontend	React 19 + Vite 8	Builds the screens. Vite runs and builds it fast.	React uses reusable components; Vite is quick to start.
Routing	react-router-dom	Moves between pages (Home, Donor, NGO, Admin) without reloading.	Standard way to build single-page apps.
Charts	Recharts	Bar, line and pie charts on dashboards.	Easy chart library for React.
Icons	lucide-react	Icons like shirt, check, clock.	Light and simple.
Backend	Node.js + Express 5	Handles API requests, rules and security.	Same language (JavaScript) as frontend; Express is simple. Express 5 catches errors in async code by itself.
Database	PostgreSQL	Stores users, listings, claims, notifications.	Data is connected (users → listings → claims), so a relational database fits. Also supports strong rules and locks.
DB tool	Knex + pg	Writes SQL queries, runs migrations, handles transactions and connection pool.	Safe parameterized queries; schema changes are tracked in migration files.
Login	jsonwebtoken (JWT)	Creates a 24-hour token holding user id and role.	Stateless: server does not need to keep sessions.
Passwords	bcryptjs	Hashes passwords (salt rounds 10). Also hashes reset codes.	Passwords are never stored as plain text.
Security	cors	Allows only trusted websites to call the API.	Protects the API in production.
Config	dotenv	Loads secrets/settings from a .env file.	Keeps secrets out of the code.
Images	Local disk / AWS S3 / Cloudinary	Saves photos. One setting (IMAGE_STORAGE_DRIVER) picks the storage.	Local is easy for development; S3/Cloudinary work in the cloud.
Auth (future)	firebase-admin	Set up to verify Firebase ID tokens later.	Planned for production login.
Backend tests	node --test (built-in)	Runs 55 integration test suites.	No extra library needed.

Part	Technology	What it does here	Simple reason
Frontend tests	Vitest + Playwright	Vitest for unit tests; Playwright for real browser journeys.	Fast unit tests and realistic end-to-end tests.
Lint	oxlint	Checks code quality.	Very fast linter.
DevOps	Git, Docker, GitHub Actions	Source control, container image, automatic test pipeline (4 jobs).	Same environment everywhere; tests run on every push.
Cloud plan	AWS Amplify, ECS Fargate, RDS, S3, CloudFront, Secrets Manager	Written in AWS_DEPLOYMENT.md as the deployment plan.	Managed services for hosting, database, storage and secrets.

Things that are NOT used (good to know)

- No multer: photos are sent as base64 text inside JSON, not as multipart file upload.
- No cron or queue library: the expiry job uses Node's built-in setInterval.
- No Jest or Mocha: backend tests use Node's built-in test runner.
- No ESLint: the project uses oxlint.
- No rate-limiting library yet (listed as an improvement).

Part 4. High-level design (HLD)



How one request travels (easy steps)

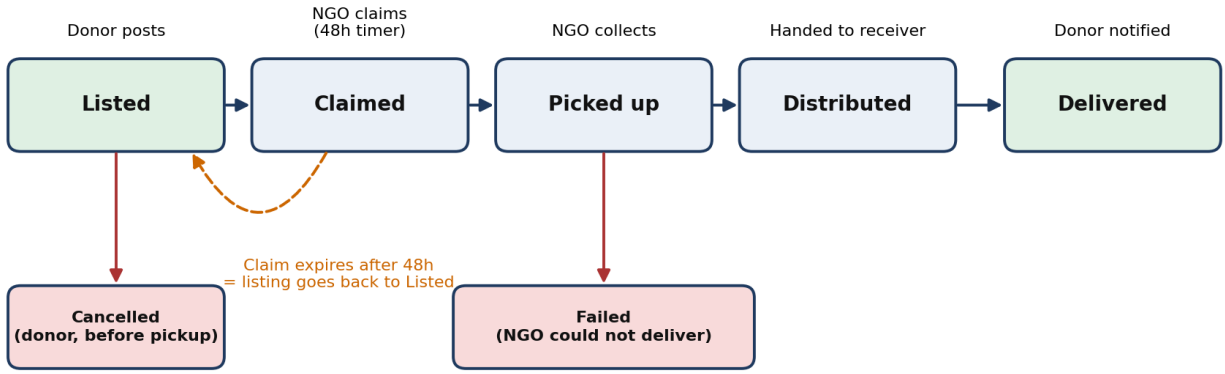
- 1. The user clicks something in the React app (for example "Claim listing").
- 2. The frontend sends an API request to the Express backend. It adds the JWT token in the header: Authorization: Bearer <token>.
- 3. Express checks the token (requireAuth), then checks the role (requireRole) and, for NGOs, checks the database to confirm they are verified (requireVerifiedNgo).
- 4. The route runs the business logic inside a database transaction, so all changes succeed together or none do.
- 5. PostgreSQL saves the data. If photos are involved, the image storage driver saves them.
- 6. The backend sends a JSON response and the frontend updates the screen.
- 7. In the background, a job runs every 5 minutes to expire old claims.

Scaling idea (if the interviewer asks)

The backend does not keep user sessions in memory (JWT is stateless), so many copies of the backend can run behind a load balancer. The expiry job uses a database lock (FOR UPDATE SKIP LOCKED), so even if every copy runs the job, no claim is processed twice. Photos must use S3 or Cloudinary in that case, because local disk is not shared. The AWS plan in the project uses ECS Fargate for the API, RDS for the database and S3 with CloudFront for images.

Part 5. Donation lifecycle (the heart of the app)

Donation Lifecycle (5 steps)



Status	Meaning	Next allowed steps
listed	Donor posted it. It is open to NGOs.	claimed, cancelled
claimed	A verified NGO claimed it. 48-hour timer starts.	picked_up, cancelled, listed (if claim expires)
picked_up	NGO collected the clothes.	distributed, failed
distributed	NGO handed clothes to a receiver.	delivered
delivered	Donation complete. Donor gets thank-you notification.	(final)
cancelled / failed	Donor cancelled, or NGO could not deliver.	(final)

A small file called listingStatusTransitions.js is the “rule book”. It lists which status changes are allowed. Any other change is rejected with HTTP 409 Conflict.

Distributed and Delivered happen together in one request (POST /claims/:id/distribute). This is because handing clothes to the receiver and finishing the delivery are the same real-world moment.

Part 6. Folder structure (low-level design)

```
Clothing-Donation-App/
|-- .github/workflows/ci.yml   GitHub Actions pipeline
|-- amplify.yml                AWS Amplify settings for the frontend
|-- AWS_DEPLOYMENT.md          Cloud deployment guide
|-- Design/  HLD.md, LLD.md    Architecture and detailed design documents
|-- Spec/    spec.md, plan.md, tasks.md   Requirements, plan and task checklist
`-- Code/
    |-- backend/
    |   |-- index.js           App start, CORS, routes, global error handler
    |   |-- knexfile.js        DB settings (dev, test, production)
    |   |-- migrations/        12 files that create/change tables
    |   |-- seeds/             Sample users and dashboard data
    |   |-- scripts/           run-tests.js, security-spot-check.js, add-admin.js
    |   |-- test/              55 integration test files
    |   |-- uploads/           Photos saved here in local mode
    |   `-- src/
    |       |-- routes/        auth, listings, claims, ngo, notifications, admin, dashboard
    |       |-- middleware/    auth.js, role.js, requireVerifiedNgo.js
    |       |-- services/      sessionToken, verifyNgo, imageStorage drivers,
    |       |                  listingStatusTransitions, staleClaims,
    |       |                  reverseGeocode, distance, passwordReset
    |       `-- jobs/          claimExpiryJob.js
    `-- frontend/
        |-- e2e/               Playwright tests
        `-- src/  App.jsx, api/client.js, context/AuthContext.jsx,
                  components/, pages/
```

Part 7. Database design

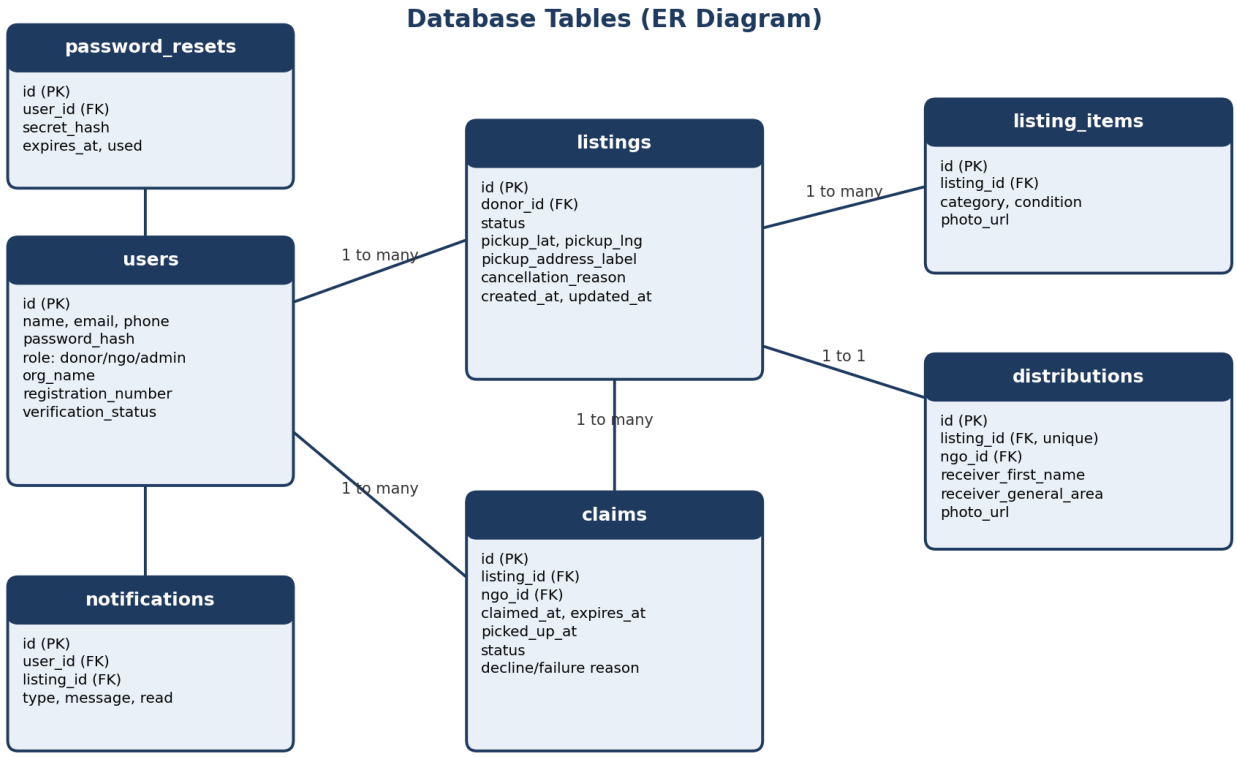


Table	What it stores	Important rules
users	All accounts: donors, NGOs, admins. NGOs also have org_name, registration_number, verification_status.	Email is unique. Registration number is unique (when present). A CHECK rule: NGO rows must have NGO fields; donor/admin rows must not.

Table	What it stores	Important rules
listings	One donation bundle: donor, status, pickup latitude/longitude, address label, cancel info.	Has indexes on status+location, donor+date, status+date for fast searching.
listing_items	Each clothing item in a bundle: category, condition, photo_url.	Deleted automatically if the listing is deleted (ON DELETE CASCADE).
claims	An NGO’s claim on a listing: claimed_at, expires_at (now + 48 hours), picked_up_at, status.	Partial unique index: only ONE active claim per listing.
distributions	Final handover: receiver first name, general area, optional photo.	listing_id is unique, so one distribution per listing.
notifications	In-app messages: user, listing, type, message, read flag.	Index on user + read + date for fast loading.
password_resets	Hashed reset code/token, method, expiry (15 min), used flag.	Single use. Secret is stored hashed, never as plain text.

Other database features used: UUID primary keys, native ENUM types (user_role, listing_status, claim_status, notification_type), foreign keys, TIMESTAMPTZ time columns, and 12 migration files so the schema is versioned.

Part 8. API endpoints (about 30)

Method + URL	Who can use	What it does
GET /health	Public	Check that server is running.
POST /auth/signup/donor	Public	Create donor account.
POST /auth/signup/ngo	Public	Create NGO account and auto-check registration number.
POST /auth/login	Public	Check password, return JWT token and user.
POST /auth/forgot-password	Public	Create 6-digit code or reset link.
POST /auth/reset-password/code and /link	Public	Set a new password.
POST /listings/upload-photo	Donor	Upload one photo (base64) and get its URL.
GET /listings/reverse-geocode	Donor	Turn latitude/longitude into an area name.
POST /listings	Donor	Create a listing with items (inside a transaction).
GET /listings/mine	Donor	See my listings.
GET /listings/open	Verified NGO	Browse open listings; filter by radius and category. GPS is hidden.
POST /listings/:id/claim	Verified NGO	Claim a listing.
POST /listings/:id/cancel	Donor (owner)	Cancel a listing with a reason.
POST /claims/:id/pickup	NGO (owner)	Mark picked up.
POST /claims/:id/distribute	NGO (owner)	Record receiver; listing becomes delivered.
POST /claims/:id/decline	NGO (owner)	Release a claim before pickup.
POST /claims/:id/fail	NGO (owner)	Report that delivery failed after pickup.
GET /ngo/me	NGO	My profile and verification status.
POST /ngo/:id/reverify	NGO (self only)	Send the registration number again for verification.
GET /ngo/claims/mine	NGO	My claim history.
GET /notifications	Any logged-in user	My notifications, unread first.
POST /notifications/:id/read	Notification owner	Mark one notification as read.
GET /admin/listings, DELETE /admin/listings/:id	Admin	View and delete listings.
GET /admin/claims/stale	Admin	View expired claims.
GET /admin/ngo-verifications	Admin	View NGO verification records.
GET /api/dashboard/donor | ngo | admin	Each role	Chart and summary data.
GET /uploads/:filename	Public	Serve saved photos (local mode).

Part 9. Login and security explained simply

Signup

- Donor: check fields → hash password with bcrypt (10 rounds) → save user → return profile without the password.
- NGO: same as donor, plus a check of the registration number using verifyNgo(). In the current code this is a mock: a number starting with APPROVED- becomes “verified”; anything else is “not_verified”. A real government registry connection is a future step.

Login

- Find the user by email. If not found or the password is wrong, return the same generic 401 message (so attackers cannot find out which emails exist).

- If correct, create a JWT with { uid, role } that expires in 24 hours.
- The frontend stores it in localStorage (key: donation-app.auth). The API client automatically adds it to every request.

Three security checks on protected routes

Check	File	What it does
requireAuth	middleware/auth.js	Reads the Bearer token, verifies it with JWT_SECRET, puts { uid, role } on the request. Returns 401 if bad or expired.
requireRole('ngo')	middleware/role.js	Returns 403 if the user's role is wrong.
requireVerifiedNgo	middleware/requireVerifiedNgo.js	Looks in the DATABASE (not the token) to see if the NGO is verified. Needed because verification can change after the token was made.

Other security points

- Ownership checks: an NGO can only act on its own claims; a donor can only cancel their own listing; a user can only mark their own notifications as read.
- POST /ngo/:id/reverify requires login + NGO role + ownership (req.user.uid must equal :id). This stops one NGO from changing another NGO's data.
- Ownership is not the same as proof of legitimacy. The ownership check only proves "this is my account"; it does not prove the registration number really belongs to that organization. That is why real government verification is planned.
- OTP by phone/email proves you control that phone/email, not that the NGO is real.
- Password reset secrets are hashed with bcrypt and expire in 15 minutes.
- Photo filenames are made by the server with crypto.randomUUID(), so users cannot control file paths.
- CORS only allows trusted origins in production.
- The global error handler returns a simple "Internal server error" message, not internal details.

Part 10. Important features explained

A. What if two NGOs claim at the same time?

There are two protection layers. Layer 1 (code): the update query says "change status to claimed WHERE status is still listed". The first request changes 1 row. The second changes 0 rows, so the server returns 409 Conflict. Layer 2 (database): a partial unique index on claims(listing_id) where status = 'active' makes it impossible to have two active claims for one listing, even if the code had a bug.

B. Claim expiry job (48 hours)

- File: jobs/claimExpiryJob.js. It uses setInterval and runs every 5 minutes (CLAIM_EXPIRY_JOB_INTERVAL_MS = 300000).
- It finds claims that are active, not picked up, and past expires_at. The query uses FOR UPDATE SKIP LOCKED, so many servers can run it without handling the same row twice.
- Each claim is handled in its own transaction: set claim to expired → move listing from claimed back to listed → notify both the NGO and the donor.
- It is safe to run twice (idempotent): the update only works if the claim is still active.

C. Notifications

- Stored in the notifications table. Types: claimed, picked_up, distributed, delivered, cancelled, expired, declined, failed.

- Created inside the same transaction as the status change, so a notification never exists without the real change.
- GET /notifications returns only my notifications, unread first, newest first.
- On distribute, two notifications are created: “distributed” and “delivered”. The delivered one gets a time +1 millisecond because PostgreSQL now() is the same for the whole transaction. Without this, their order on screen would be unclear.

D. Image upload

- The frontend turns the photo into a base64 Data URI and sends it in JSON (Express limit set to 8 MB).
- The backend checks the format with a pattern, allows only png, jpeg, webp and gif, and limits the size to 5 MB.
- It saves the file with a random UUID name using the chosen driver: local disk (/uploads/…), AWS S3 (private bucket, encrypted, served through CloudFront) or Cloudinary.
- Because of base64, the data is about 33% bigger. This is the trade-off for not needing multer.

E. Privacy by design

- Open listings sent to NGOs have pickup_lat and pickup_lng removed. NGOs see only the area name and distance.
- The distributions table saves only the receiver’s first name and general area, not full names or addresses.

F. Location and distance

- Distance between NGO and listing is calculated with the Haversine formula (in km).
- Reverse geocoding works offline by comparing the coordinates with 5 Bangalore reference areas; far away points show “Near lat, lng”. This avoids depending on an outside API.

Part 11. Testing

- Backend: 55 integration test suites with Node’s built-in runner (node --test). They cover signup/login, role checks, listing creation and cancel, open listings (distance, category, masking), claim (including a race condition test), pickup, distribute (including rollback), claim expiry, NGO verification, admin routes, image storage, notifications and the state machine.
- Test isolation: scripts/run-tests.js uses a separate TEST_DATABASE_URL, creates the test database if missing, runs migrations, and truncates all tables before each run. Tests never touch development data.
- Security spot check: scripts/security-spot-check.js confirms that unauthorized calls return 403.
- Frontend unit tests: Vitest, 6 tests for the API client (URL building, headers, token, errors).
- Frontend end-to-end tests: Playwright journeys for app load, donor (signup, photo, location, listing) and NGO (signup, verify, browse, claim, pickup, distribute).
- CI: GitHub Actions runs four parallel jobs (backend, security, frontend, e2e) with a PostgreSQL service.
- Manual testing: you ran a 20-step local test in the browser. The donor received four notifications in order (claimed → picked_up → distributed → delivered) and the logs were clean.
- A coverage tool is not set up. Do not quote a coverage percentage.

Part 12. Problems we faced and how we fixed them

Problem	Why it happened	What we did	What I learned
Login gave Internal Server Error; database connection refused	The backend tried port 5433, but PostgreSQL was listening on 5432.	Found the mismatch and set DATABASE_URL to the real port.	Always check the real listening port.
Forgot the local PostgreSQL password	Local postgres password was unknown.	Planned a safe reset (temporary trust login, change password, then restore secure scram-sha-256 setting).	Always restore secure login settings after a reset.
Could not stop the PostgreSQL service in PowerShell	Needed administrator rights.	Run PowerShell as Administrator.	Service commands may need elevated rights.
NGO could re-verify another NGO's record	Route had no ownership check.	Added requireAuth + NGO role + req.user.uid === id; updated tests.	Check the role AND who owns the resource.
NGO verification status missing after login	Login response did not include verification_status.	Added GET /ngo/me and fetched it in AuthContext.	Fetch live profile data when it can change.
Two notifications had the same time	PostgreSQL now() is fixed inside a transaction.	Gave the delivered notification a +1 ms time.	Be careful with timestamps inside transactions.
Race condition: two NGOs claiming the same listing	Two requests at the same moment.	Optimistic status update (WHERE status='listed') plus partial unique index; test listings-claim-race.	Protect important rules in the database, not only in code.
Same claim processed twice by the expiry job	Several workers or repeated runs.	FOR UPDATE SKIP LOCKED and an idempotent update.	Design background jobs to be safe to repeat.
Unread badge stayed after opening notifications	Only the clicked item was marked read.	A prompt was prepared to mark all as read when the dropdown opens.	Understand how read-state is saved.

Part 13. How to run the project

```
# 1) Database (PostgreSQL, default port 5432)
# 2) Backend
cd Code/backend
npm install
npm run migrate:up      # create tables
npm run seed            # add sample users/data
npm start               # runs on http://localhost:4000
# check: http://localhost:4000/health  ->  {"status":"ok"}

# 3) Frontend
cd Code/frontend
npm install
npm run dev             # runs on http://localhost:5173

# 4) Tests
cd Code/backend  && npm test && npm run security-check
cd Code/frontend && npm test && npm run lint && npm run test:e2e
```

Important settings in .env (names only): PORT, DATABASE_URL, JWT_SECRET, CORS_ALLOWED_ORIGINS, IMAGE_STORAGE_DRIVER, CLAIM_EXPIRY_JOB_INTERVAL_MS, TEST_DATABASE_URL. For cloud: S3_BUCKET_NAME, S3_PUBLIC_BASE_URL, AWS_REGION, Cloudinary keys, Firebase keys. Frontend: VITE_API_BASE_URL.

Part 14. Limitations and what I would improve

- Add rate limiting on login, forgot-password and upload routes (currently none).
- Replace the mock NGO check with a real government registry API.
- Send real email/SMS for password reset (now it only prints to the console; in development the code is returned in the response for easy testing).
- Normalize emails (trim and lowercase) and validate phone numbers (for example E.164 format).
- Use a real geocoding service instead of 5 hard-coded areas.
- Move the expiry job to a proper queue (for example BullMQ or SQS) when running many servers.
- Use S3 or Cloudinary in production, because local disk is lost when a container restarts.
- Add a code coverage tool.
- Finish production deployment on AWS and use Firebase Auth if needed.

Part 15. Interview questions and easy answers

Q1. Tell me about your project.

WearForward connects clothing donors with verified NGOs. A donor lists clothes with photos and location. A verified NGO claims it, picks it up and hands it to a receiver. The donor gets notified at each step.

Q2. What problem does it solve?

People have clothes they do not use, and NGOs need them. The app makes it easy to connect them and tracks the whole journey from listing to delivery.

Q3. Who are the users?

Donors, NGOs and Admins. Admins cannot sign up from the app; they are created by a script or in the database, to stop people from making themselves admin.

Q4. Explain your tech stack.

React + Vite on the frontend, Node.js + Express 5 on the backend, PostgreSQL with Knex for data, JWT and bcrypt for security. Tests use Node’s test runner, Vitest and Playwright. Docker and GitHub Actions are used for packaging and CI.

Q5. Why PostgreSQL and not MongoDB?

The data is connected: users, listings, items, claims, notifications. PostgreSQL gives foreign keys, transactions, enums and special indexes. I used a partial unique index and row locking, which are strong features of PostgreSQL.

Q6. Explain the architecture.

The browser runs the React app. It calls the Express API with a JWT token. Express checks login and role, runs logic in transactions on PostgreSQL, and stores photos using a storage driver. A background job expires old claims.

Q7. Explain the donation lifecycle.

Listed → Claimed → Picked up → Distributed → Delivered. A listing can also be Cancelled by the donor, or Failed if the NGO cannot deliver. A small state-machine file lists the allowed moves and rejects the rest.

Q8. What happens when an NGO claims a listing?

In one transaction: the listing changes from listed to claimed, a claim row is created with expiry in 48 hours, and a notification is added for the donor. If another NGO claimed first, the update changes 0 rows and the API returns 409.

Q9. What if two NGOs claim at the same time?

Only one wins. The code update uses WHERE status = 'listed'. Also the database has a partial unique index allowing only one active claim per listing. So there are two layers of protection.

Q10. How does claim expiry work?

A setInterval job runs every 5 minutes. It finds active claims past their 48-hour time and not picked up. It marks them expired, puts the listing back to listed, and notifies both sides. It uses FOR UPDATE SKIP LOCKED so many servers do not process the same claim.

Q11. Why FOR UPDATE SKIP LOCKED?

It locks the rows I am working on and lets other workers skip them instead of waiting. This avoids duplicates and deadlocks when the API runs on multiple servers.

Q12. How does login work?

The user sends email and password. The server compares the password with the bcrypt hash. If it matches, it signs a JWT with uid and role, valid for 24 hours. The frontend stores it and sends it as a Bearer token on each request.

Q13. Why does requireVerifiedNgo check the database and not the token?

A token lives for 24 hours. If an NGO’s status changes in that time, the token would be out of date. So I only trust the user id from the token and read the live status from the database.

Q14. How do you verify NGOs?

At signup the server calls verifyNgo(). Right now it is a mock: registration numbers starting with APPROVED- are verified. A real government API is a planned improvement. There is no admin override on purpose.

Q15. What security did you add to the reverify route?

Login, NGO role, and an ownership check (req.user.uid must equal the id in the URL). It stops one NGO from editing another NGO.

Q16. Does OTP prove an NGO is real?

No. OTP only proves they control a phone or email. To prove the NGO is real you need to check an official registry.

Q17. How do you upload images without multer?

The frontend converts the image to a base64 Data URI and sends it in JSON. The backend checks type (png, jpeg, webp, gif) and size (5 MB), saves it with a random UUID name, and returns a URL. The trade-off is about 33% extra size.

Q18. How are images stored?

Using a driver chosen by IMAGE_STORAGE_DRIVER: local disk for development, AWS S3 with CloudFront or Cloudinary for the cloud.

Q19. How do notifications work?

Rows in a notifications table, created in the same transaction as the status change. Each user fetches only their own, unread first. On distribute there are two notifications, and the second gets +1 ms so the order is correct.

Q20. How do you protect donor privacy?

Open listings do not include exact latitude and longitude. NGOs see only the area name and distance. Receiver data is limited to first name and general area.

Q21. How do you test the app?

55 backend integration suites (including a claim race test and a rollback test), Vitest unit tests, Playwright browser tests, a security spot-check script, and a GitHub Actions pipeline. Tests run on a separate test database that is cleaned before each run.

Q22. What was the hardest problem you faced?

Database connection refused because the backend used port 5433 while PostgreSQL was on 5432. I checked which port PostgreSQL was really using and fixed the DATABASE_URL. Another hard one was protecting against two NGOs claiming at once.

Q23. How would you scale the backend?

JWT is stateless, so I can run many copies of the API behind a load balancer. The expiry job is safe with SKIP LOCKED. Images must go to S3 or Cloudinary. The database connection pool size is set with DATABASE_POOL_MAX.

Q24. What are the weak points of the project?

No rate limiting yet, mock NGO verification, reset codes only printed to console, basic email and phone validation, and only 5 hard-coded areas for reverse geocoding.

Q25. What would you improve next?

Real NGO registry check, rate limiting, real email/SMS, proper job queue, production AWS deployment, and a coverage report.

Q26. What was your personal contribution?

Say clearly which parts you coded or tested yourself. You can honestly say you built and debugged the app with Claude Code, ran the end-to-end tests, and understand every flow in this guide.

Part 16. Quick revision sheet (read this just before the interview)

Stack in one line

React 19 + Vite | Node.js + Express 5 | PostgreSQL + Knex | JWT + bcrypt | Local/S3/Cloudinary images | Vitest + Playwright + node --test | Docker + GitHub Actions.

Architecture in 3 lines

- Browser (React) → Express API (JWT, roles, transactions) → PostgreSQL.
- Photos go to a storage driver: local disk, S3 or Cloudinary.
- A background job (setInterval, every 5 min) expires claims older than 48 hours.

Numbers to remember

Item	Value
Donation steps	5: listed, claimed, picked_up, distributed, delivered
Claim expiry	48 hours; job runs every 5 minutes (300000 ms)
JWT lifetime	24 hours
Password hashing	bcrypt, 10 salt rounds
Reset code/link lifetime	15 minutes
Photo limit	5 MB per image; JSON body limit 8 MB
Backend tests	55 integration suites
Database tables	7
Migrations	12
Ports	Frontend 5173, Backend 4000, PostgreSQL 5432

Top 5 points to say proudly

- Two-layer protection against double claims (code + database index).
- Safe background job with SKIP LOCKED and idempotent updates.
- Everything that changes many tables happens in a transaction.
- Privacy: hidden GPS and minimal receiver data.
- Strong testing: 55 backend suites, Playwright, security check, CI pipeline.

Be honest about these

- NGO verification is a mock for now.
- Reset code delivery is a console stub.
- AWS deployment is documented as a plan; do not say it is live unless you deployed it.
- Firebase Auth is set up in code but not the login method used now.
- No code coverage tool and no rate limiting yet.

If you do not know something, say: “I would need to check the code for the exact detail, but here is how the idea works.” Then explain the idea. This sounds honest and confident.